

WouldBeRows Render & the Deadline Timeline

How the interactive office card is built — the hover swap, the lazy goal fetch, and the deadline timeline (positioning, spacing, the "you" marker, the goal tag). Read this alongside the practice problems in this folder; each section maps to one.

1. The big picture

Every office in the WouldBe feed is a card. It has two faces:

- **Collapsed** (`.wouldBeCard`) — a compact summary: the headline deadline + office name.
- **Hovered** (`.hoveredWouldBeCard`) — the same fixed size, but showing a **timeline** of every deadline, a **Today** marker, and the **recommended financing goal**.

The data pipeline behind the hovered view is the heart of it:

```
office.deadlines → group by date → merge same-day → fold in Today → space evenly → (fetch goal) → render
```

The golden rule you'll see repeated: **build the view-model first, then let the JSX be a dumb** `.map`. All the math (grouping, spacing, formatting) happens before render; the component only reads `p.left`, `p.labels`, `model.goalLabel`.

2. Hover architecture (problem: react/HoverCard)

The card is its **own component** with its own state. That matters:

Rules of Hooks. If you rendered a card by *calling* `renderCard(office)` inside a `.map`, every `useState` inside it would count against the *parent's* hook list — and since the number of cards changes, React throws *"rendered more hooks than during the previous render."* A real `<HoverCard />` **element** gives each card its own isolated hook slot. Always render components as `<Card />`, never `Card()`.

```
const [hovered, setHovered] = useState(false);

<div
  className={hovered ? "hoveredWouldBeCard" : "wouldbeCard"}
  onMouseEnter={handleEnter}
  onMouseLeave={() => setHovered(false)}>
  {hovered ? (/* expanded: name + <DeadlineTimeline/> */)
    : (/* collapsed: just the name */)}
</div>
```

`hovered ? A : B` is a **ternary** because it's genuinely "expanded *or* collapsed." Contrast with `cond && A`, which is "show *A or nothing*" — that's what we use for the goal tag (it hides when there's no goal).

Lazy goal fetch — a tri-state

We don't fetch a goal for every card up front; we fetch the **first time a card is hovered**. The goal state has **three** values, not two:

goalCents value	Meaning	Renders
<code>undefined</code>	not fetched yet	nothing (yet)
<code>null</code>	fetched, none on record	no tag
a number (cents)	a real goal	the tag

```
async function handleEnter() {
  setHovered(true);
  if (goalCents !== undefined) return; // already fetched -> don't re-fetch
  try {
    const res = await api.get(`/api/offices/${office.id}/recommended-goal`);
    setGoalCents(res.data?.recommended_goal_cents ?? null);
  } catch { setGoalCents(null); }
}
```

The `undefined` sentinel is exactly what the `if (... !== undefined) return` guard keys off, so the network call fires once no matter how many times you hover.

3. Building the deadline points (problems 01, 06)

An office arrives with a flat `deadlines: [{ type, date }]`. Two transforms turn that into timeline points.

a) Group + merge by date (problem 01)

Two deadlines on the same day (e.g. *Petition due* and *Filing closes* both on Aug 7) must become **one** tick. A `Map` keyed by date does it:

```
const byDate = new Map();
for (const d of deadlines) {
  const label = DEADLINE_LABELS[d.type] ?? d.type; // human name, fallback to raw
  if (byDate.has(d.date)) byDate.get(d.date).labels.push(label);
  else byDate.set(d.date, { date: d.date, labels: [label] });
}
```

First sighting of a date `set` s a fresh entry; every later same-day deadline `get` s it and `push` es another label. `DEADLINE_LABELS` maps machine types (`filing_close`) to readable text (*Filing closes*); `?? d.type` stops an unmapped type from becoming `undefined`.

b) Fold in "Today" (problem 06)

Today is **just another point**, so the same positioning code handles it. If a deadline already falls on today's date, we flag that entry; otherwise we add a new point with an **empty** labels array:

```
if (byDate.has(today)) byDate.get(today).isToday = true;
else byDate.set(today, { date: today, labels: [], isToday: true });

const points = [...byDate.values()].sort((a, b) => a.date.localeCompare(b.date));
```

The empty `labels: []` is a signal. At render time, empty $\Rightarrow$ draw the "Today" text + the red "you" badge; non-empty $\Rightarrow$ draw the deadline name(s) + date. ISO dates (`"YYYY-MM-DD"` , zero-padded) sort correctly as plain strings — no date parsing needed.

4. Spacing the deadlines — three designs (problems 02, 05, 03)

This is the part we iterated on the most. Three approaches, in the order we tried them:

Design 1 — Proportional (problem 02)

Place each date by its *real* position between the earliest and latest, so gaps reflect actual elapsed time:

```
pos% = (thisDate - min) / (max - min) × 100
```

Honest, but when dates cluster (the fall FEC-report dates), the labels bunch up and **overlap**. Dates only subtract as numbers, so convert with `new Date(`${iso}T00:00:00`).getTime()`. The `T00:00:00` forces *local* midnight — the offset cancels in the ratio and it dodges the " `new Date('2026-01-01')` is UTC, shows Dec 31 in the US" bug. Guard `max === min` (single date) to avoid dividing by zero.

Design 2 — Collision stagger (problem 05)

Keep proportional positions, but push labels that would collide down to lower rows. Greedy sweep, left to right: each label takes the highest row whose last label is $\geq \text{minSep}$ away, else opens a new row.

```
const rowLastCenter = [];  
const rowOf = positions.map((c) => {  
  for (let r = 0; r < rowLastCenter.length; r++)  
    if (c - rowLastCenter[r] >= minSep) { rowLastCenter[r] = c; return r; }  
  rowLastCenter.push(c); return rowLastCenter.length - 1;  
});
```

The bug this taught us. Staggering fixes *horizontal* overlap, but two rows still collide *vertically* unless each row's vertical drop **exceeds a label's height**. Our first attempt dropped rows only 15px while labels were ~30px tall — still overlapping. You cannot catch this without actually **rendering it** (we screenshotted a headless-Chrome preview to see it). Lesson: for layout, build a visual preview, don't trust the code compiling.

Design 3 — Even spacing (what we shipped, problem 03)

Give up strict proportionality. Space the points **evenly** in date order so, with every name + date always shown, no two labels can overlap — and the card stays a **fixed size**.

```
left(i) = n === 1 ? 50 : (i / (n - 1)) * 100    // n points make n-1 gaps
```

Design	Spacing reflects...	Overlap?	Card size	Verdict
Proportional	real time between dates	yes, when clustered	varies	rejected
Stagger	real time (multi-row)	no, if rows tall enough	grows tall	rejected
Even	order only	never	fixed	shipped

5. Positioning in CSS — the layout core

The track is `position: relative`; every tick is `position: absolute`. The key pairing:

```
.wouldbeTick {
  position: absolute;
  left: {p.left}%;          /* set inline per point */
  transform: translateX(-50%); /* slide back by half its own width */
}
```

Why both? `left: x%` puts the tick's *left edge* at the percent. `translateX(-50%)` then shifts it back half its width so the **line** — the tick's center — lands exactly on the date. Drop the transform and every label sits to the right of its mark.

The pieces

Element	How it's placed
Axis line (<code>.wouldbeTimelineLine</code>)	<code>position:absolute; left:0; right:0; top:13px; height:1px</code> — one horizontal rule.
Tick mark	a 1px-wide vertical line dropping from the axis to its label.
Label	a flex column under the mark: name line(s) on top, date beneath, small font.
"you" badge	<code>position:absolute; bottom:100%</code> — floats <i>above</i> the axis regardless of tick height.
Goal tag	<code>position:absolute; top; right</code> on the card — costs zero vertical space, so the fixed-height card isn't pushed.

Because the card is a fixed height (same as the collapsed card) with `overflow: hidden`, pinning the goal tag with absolute positioning (instead of a flex row) is what lets everything fit.

6. The two formatting helpers

```
// "2026-08-07" -> "Aug 7, 2026". T00:00:00 = local midnight, dodges the TZ off-by-one.
function formatDeadlineDate(iso) {
  return new Date(`${iso.slice(0,10)}T00:00:00`)
    .toLocaleDateString(undefined, { year:"numeric", month:"short", day:"numeric" });
}

// 25000000 (cents) -> "$250,000". Store cents, format only at the edge.
function formatUSD(cents) {
  return (cents / 100).toLocaleString("en-US",
    { style:"currency", currency:"USD", maximumFractionDigits: 0 });
}
```

Money is always an **integer of cents** in storage (floats can't hold `$0.10` exactly); we divide and format only for display. `Intl` handles the `$`, commas, and rounding — never hand-roll it.

7. The capstone model (problem 07)

Everything above assembles into one function whose output the JSX maps over:

```
function buildTimeline({ deadlines, today, goalCents }) {
  const points = buildPoints(deadlines, today);           // group + merge + Today
  const n = points.length;
  return {
    goalLabel: goalCents == null ? null : formatUSD(goalCents), // == folds null+undefined
    points: points.map((p, i) => ({ ...p, left: evenPct(i, n) })),
  };
}
```

Formatting and layout live in the **model**, not the render — so the component stays a dumb map over `points`, reading `p.left`, `p.labels`, `p.isToday`, and `model.goalLabel`. That separation is the whole point: **logic is testable in plain Node (the `.js` problems); the JSX is just the shell.**

Take-away checklist

- Card = its own component (own hooks) — render `<Card/>`, never `Card()`.
- Hover = `useState` + `onMouseEnter/Leave` + a ternary.
- Goal = lazy fetch on first hover, tri-state (`undefined` / `null` / number).
- Points = `Map` grouped by date, same-day merged, Today folded in.
- Spacing = even (order), not proportional (dates) — no overlap, fixed card.
- Position = `left:% + translateX(-50%)`; badge `bottom:100%`; goal `absolute`.
- Money = integer cents, `Intl` at the edge. Dates = `T00:00:00` to dodge TZ.
- Layout can't be verified by "it compiles" — render and look.